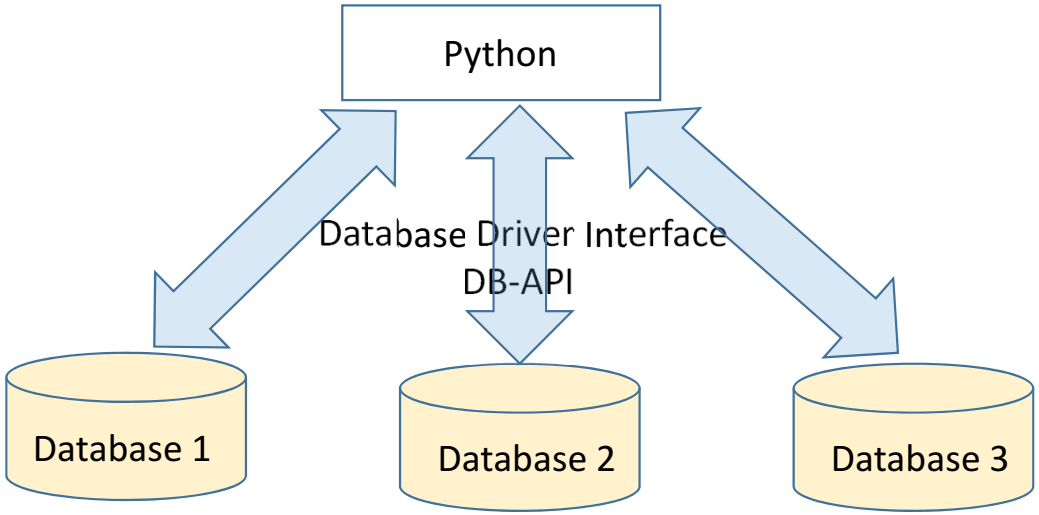
SQLite

Overview – DB-API



Database	Python Package
Oracle	cx_oracle, pyodbc
SQL Server	pymssql, pyodbc
PostgreSQL	psycopg2
MySQL	MySQL Connector/Python, pymysql

Overview – DB-API

Python » 3.7.9 Documentation » The Python Standard Library » Data Persistence »

sqlite3 — DB-API 2.0 interface for SQLite databases

Source code: Lib/sqlite3/

SQLite is a C library that provides a lightweight disk-based database that doesn't require a separate server process and allows accessing the database using
Some applications can use SQLite for internal data storage. It's also possible to prototype an application using SQLite and then port the code to a larger databa

The sqlite3 module was written by Gerhard Häring. It provides a SQL interface compliant with the DB-API 2.0 specification described by **PEP 249**.

To use the module, you must first create a `Connection` object that represents the database. Here the data will be stored in the `example.db` file:

```
import sqlite3
conn = sqlite3.connect('example.db')
```

Steps

- Import sqlite3 module

```
import sqlite3
```

- Get connection object (Connection Establishment)

```
con = sqlite3.connect('library.db')
```

  - create a connection to the database (library.db) in the current directory
  - if library.db file does not exists, a new file will be created

- Set row_factory attribute of Connection object

```
con.row_factory = sqlite3.Row
```

  - by default, rows are returned as tuples – type(row) will returns tuple class. Column can be access by index (position of the field).
  - Setting the row_factory to sqlite3.Row - each row is retrieved as a dictionary mapping column names to values. Columns can be access by name or index.

Accessing columns by name instead of by index

Steps

- Get cursor object

```
cur = con.cursor()
```

  - database cursor is an identifier associated with a group of rows. It is a pointer to the current row in a buffer.
  - pointer to retrieve, move through and process each row of a result set from a query.

- Execute SQL statement

```
resultset = cur.execute('SELECT Name FROM STUDENT')
cur.execute('CREATE TABLE ABC…')
```

Steps

- Iterating the resultset (with row_factory attribute set to sqlite3.Row)

```
result = cur.execute('SELECT Name FROM STUDENT')
rows = result.fetchall()
for rec in rows:
    print(rec['Name'])
```

Steps

- Commit/Rollback transaction (Transaction Management)

```
con.commit()
con.rollback()
```

  - Use connection object

- Close cursor/connection

```
cursor.close()
con.close()
```

Steps

```
import sqlite3
con = sqlite3.connect('school.db') ##open connection
con.row_factory = sqlite3.Row   ##set row factory
cursor = con.cursor()         ## get cursor and execute SQL statement
resultset= cursor.execute('SELECT Name FROM STUDENT WHERE ID=?',(12,))
records = resultset.fetchall()   ## fetch results of query
cursor.close()                       ##close cursor
for rec in records:
   print(rec['Name'])

cursor = con.cursor()
cursor.execute('UPDATE STUDENT SET Name= ? WHERE ID=? ', ('John', 12))
con.commit()     ## commit changes/update
cursor.close()
con.close()                  ##close connection
```

Prepared statements

- Using question marks placeholders

```
cur.execute(
"insert into people (name, age) values (?, ?)",
(who, age))
```

- Using named placeholders

```
cur.execute(
  "select * from people where name=:who and age=:age",
{"who": who, "age": age})
```

- **DO NOT** use String Concatenation

```
 f"Select * from people where name={name} and age={age}"
"Select * from people where name="+name+" and age="+age
```